

МОСКОВСКИЙ АВИАЦИОННЫЙ ИНСТИТУТ  
(НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ)  
Институт №8 «Компьютерные науки и прикладная математика»

**Лабораторная работа №1**  
**по курсу «Теоретическая механика и**  
**компьютерное моделирование»**

**Динамика системы**

Выполнил(а) студент группы М8О-206Б-21

Орусский В.Р.

Преподаватель: Чекина Евгения Алексеевна

Москва, 2022

## Лабораторная работа №1

Код лабораторной работы №1:

```
import numpy as np
import matplotlib
import matplotlib.pyplot as plt
from matplotlib.animation import FuncAnimation
import sympy as sp
import math

matplotlib.use('TkAgg')

# Вариант №15:  $r(t) = 2 + \sin(12t)$ ,  $\varphi(t) = 1.8t + 0.2\cos(12t)$ 

# Функция поворота на угол альфа
def Rot2D(X, Y, Alpha):
    RX = X * np.cos(Alpha) - Y * np.sin(Alpha)
    RY = X * np.sin(Alpha) + Y * np.cos(Alpha)
    return RX, RY

# Символьная переменная t (время)
t = sp.Symbol('t')

# Прописываем наш радиус-вектор и изменение угла поворота
fi = 1.8 * t + 0.2 * sp.cos(12 * t)
r = 2 + sp.sin(12 * t)

# Полярные координаты -> декартовы ( $x = \cos(r) * \cos(\varphi)$ ;  $y = \cos(r) * \sin(\varphi)$ )
x = sp.cos(r) * sp.cos(fi)
y = sp.cos(r) * sp.sin(fi)

# Вычисление скорости по x
Vx = sp.diff(x, t)

# Вычисление скорости по y
Vy = sp.diff(y, t)

# Вычисление общей скорости
Vmod = sp.sqrt(Vx * Vx + Vy * Vy)

# Вычисление ускорения по x
Wx = sp.diff(Vx, t)

# Вычисление ускорения по y
Wy = sp.diff(Vy, t)

# Вычисление общего ускорения
Wmod = sp.sqrt(Wx * Wx + Wy * Wy)
# Вычисление тангенсального ускорения
Wtau = sp.diff(Vmod, t)
# Вычисление нормального ускорения
Wnorm = sp.sqrt(Wmod * Wmod - Wtau * Wtau)
# Вычисление радиуса кривизны
rho = (Vmod * Vmod) / sp.sqrt(Wmod * Wmod - Wtau * Wtau)

A = sp.acos(Wnorm / Wmod)
B = sp.acos(Wtau / Wmod)

Wxt = Vx * (Wtau / Vmod)
```

```

Wyt = Vy * (Wtau / Vmod)
k = (Wtau / Vmod)

# Равномерное распределение по массиву тысячи чисел от 0 до 10.
T = np.linspace(0, 10, 1000)
# Заполнение массивов нулями, в соответствии с массивом T
X = np.zeros_like(T)
Y = np.zeros_like(T)
VX = np.zeros_like(T)
VY = np.zeros_like(T)
WY = np.zeros_like(T)
WX = np.zeros_like(T)
WYT = np.zeros_like(T)
WXT = np.zeros_like(T)
Rho = np.zeros_like(T)
Phi = np.zeros_like(T)
K = np.zeros_like(T)

# Меняем переменную x на t, а t меняем на значение из T[i]. По сути,
подстановка значения в символьную переменную
for i in np.arange(len(T)):
    X[i] = sp.Subs(x, t, T[i])
    Y[i] = sp.Subs(y, t, T[i])
    VX[i] = sp.Subs(Vx, t, T[i])
    VY[i] = sp.Subs(Vy, t, T[i])
    WY[i] = sp.Subs(Wy, t, T[i])
    WX[i] = sp.Subs(Wx, t, T[i])
    WYT[i] = sp.Subs(Wyt, t, T[i])
    WXT[i] = sp.Subs(Wxt, t, T[i])
    Rho[i] = sp.Subs(rho, t, T[i])
    Phi[i] = sp.Subs(fi, t, T[i])
    K[i] = sp.Subs(k, t, T[i])

# Создать окно
fig = plt.figure()
# ax1 - окно с графиком
ax1 = fig.add_subplot(1, 1, 1)
ax1.axis('equal')
ax1.set_title("Модель движения точки")
ax1.set_xlabel('Ось абсцисс (X)')
ax1.set_ylabel('Ось ординат (Y)')

# Построение траектории
ax1.plot(X, Y)

# Построение точки
P, = ax1.plot(X[0], Y[0], marker='o')

WTLine, = ax1.plot([X[0], X[0] + WXT[0]], [Y[0], Y[0] + WYT[0]], 'yellow',
label='Вектор tan ускорения')
VLine, = ax1.plot([X[0], X[0] + VX[0]], [Y[0], Y[0] + VY[0]], 'r',
label='Вектор скорости')
WNLine, = ax1.plot([X[0], X[0] + abs(K[0]) * (VY[0] * Rho[0] /
math.sqrt(VX[0] ** 2 + VY[0] ** 2))],
[Y[0], Y[0] - abs(K[0]) * (VX[0] * Rho[0] /
math.sqrt(VX[0] ** 2 + VY[0] ** 2))], 'g',
label='Вектор norm ускорения')
Rholine, = ax1.plot([X[0], X[0] + VY[0] * Rho[0] / math.sqrt(VX[0] ** 2 +
VY[0] ** 2)],
[Y[0], Y[0] - VX[0] * Rho[0] / math.sqrt(VX[0] ** 2 +
VY[0] ** 2)], 'brown', label='Вектор кривизны')
RLine, = ax1.plot([0, X[0]], [0, Y[0]], 'b', label='Радиус-вектор')

```

```

R = math.sqrt(math.pow(X[0], 2) + math.pow(Y[0], 2))

# Построение стрелочек на концах векторах
ArrowX = np.array([-0.2 * R, 0, -0.2 * R])
ArrowY = np.array([0.1 * R, 0, -0.1 * R])
RArrowX, RArrowY = Rot2D(ArrowX, ArrowY, math.atan2(VY[0], VX[0]))
VArrow, = ax1.plot(RArrowX + X[0] + VX[0], RArrowY + Y[0] + VY[0], 'r')

ArrowWXT = np.array([-0.2 * R, 0, -0.2 * R])
ArrowWYT = np.array([0.1 * R, 0, -0.1 * R])
RArrowWXT, RArrowWYT = Rot2D(ArrowX, ArrowY, math.atan2(WYT[0], WXT[0]))
WTAarrow, = ax1.plot(RArrowWXT + X[0] + WXT[0], RArrowWYT + Y[0] + WYT[0],
'yellow')

ArrowWXN = np.array([-0.2 * R, 0, -0.2 * R])
ArrowWYN = np.array([0.1 * R, 0, -0.1 * R])
RArrowWXN, RArrowWYN = Rot2D(ArrowWXN, ArrowWYN,
math.atan2(abs(K[0]) * (VX[0] * Rho[0] /
math.sqrt(VX[0] ** 2 + VY[0] ** 2)),
abs(K[0]) * (VY[0] * Rho[0] /
math.sqrt(VX[0] ** 2 + VY[0] ** 2))))
WNarrow, = ax1.plot(RArrowWXN + X[0] + abs(K[0]) * (VY[0] * Rho[0] /
math.sqrt(VX[0] ** 2 + VY[0] ** 2)),
RArrowWYN + Y[0] + abs(K[0]) * (VX[0] * Rho[0] /
math.sqrt(VX[0] ** 2 + VY[0] ** 2)), 'green')

ArrowRx = np.array([-0.2 * R, 0, -0.2 * R])
ArrowRy = np.array([0.1 * R, 0, -0.1 * R])
RArrowRx, RArrowRy = Rot2D(ArrowRx, ArrowRy, math.atan2(Y[0], X[0]))
RArrow, = ax1.plot(RArrowRx + X[0], RArrowRy + Y[0], 'black')

ArrowRhoX = np.array([-0.2 * R, 0, -0.2 * R])
ArrowRhoY = np.array([0.1 * R, 0, -0.1 * R])
RArrowRhoX, RArrowRhoY = Rot2D(ArrowRhoX, ArrowRhoY, math.atan2(- VX[0] *
Rho[0] / math.sqrt(VX[0] ** 2 + VY[0] ** 2),
VY[0] *
Rho[0] / math.sqrt(VX[0] ** 2 + VY[0] ** 2)))
ArrowRho, = ax1.plot(RArrowRhoX + X[0] + VY[0] * Rho[0] / math.sqrt(VX[0] **
2 + VY[0] ** 2),
RArrowRhoY + Y[0] - VX[0] * Rho[0] / math.sqrt(VX[0] **
2 + VY[0] ** 2), 'b')

# Вывод легенды на график
ax1.legend(
ncol=2, # количество столбцов
facecolor='oldlace', # цвет области
edgecolor='r', # цвет крайней линии
)
ax1.set(xlim=[-4, 4], ylim=[-4, 4])

# Функция для анимации
def anima(i):
RhoX = X[i] + VY[i] * Rho[i] / math.sqrt(VX[i] ** 2 + VY[i] ** 2)
RhoY = Y[i] - VX[i] * Rho[i] / math.sqrt(VX[i] ** 2 + VY[i] ** 2)
WXX = X[i] + abs(K[i]) * VY[i] * Rho[i] / math.sqrt(VX[i] ** 2 + VY[i] **
2)
WYY = Y[i] - abs(K[i]) * VX[i] * Rho[i] / math.sqrt(VX[i] ** 2 + VY[i] **
2)

P.set_data(X[i], Y[i])
WTLine.set_data([X[i], X[i] + WXT[i]], [Y[i], Y[i] + WYT[i]])
VLine.set_data([X[i], X[i] + VX[i]], [Y[i], Y[i] + VY[i]])
WNLine.set_data([X[i], WXX], [Y[i], WYY])
Rholine.set_data([X[i], RhoX], [Y[i], RhoY])

```

```

RLine.set_data([0, X[i]], [0, Y[i]])

RArrowX, RArrowY = Rot2D(ArrowX, ArrowY, math.atan2(VY[i], VX[i]))
RArrowWXT, RArrowWYT = Rot2D(ArrowWXT, ArrowWYT, math.atan2(WYT[i],
WXT[i]))
RArrowRx, RArrowRy = Rot2D(ArrowRx, ArrowRy, math.atan2(Y[i], X[i]))
RArrowRhoX, RArrowRhoy = Rot2D(ArrowRhoX, ArrowRhoY, math.atan2(-Y[i] +
RhoY, -X[i] + RhoX))
RArrowWXN, RArrowWYN = Rot2D(ArrowWXN, ArrowWYN, math.atan2(-Y[i] + WYY,
-X[i] + WXX))

ArrowRho.set_data(RArrowRhoX + RhoX, RArrowRhoy + RhoY)
VArrow.set_data(RArrowX + X[i] + VX[i], RArrowY + Y[i] + VY[i])
WTArrow.set_data(RArrowWXT + X[i] + WXT[i], RArrowWYT + Y[i] + WYT[i])
WNArrow.set_data(RArrowWXN + WXX, RArrowWYN + WYY)
RArrow.set_data(RArrowRx + X[i], RArrowRy + Y[i])

return P, WTLine, WTArrow, VLine, WNLine, Rholine, VArrow, RLine, RArrow,
ArrowRho, WNArrow

# Анимация
anim = FuncAnimation(fig, anima, frames=1000, interval=35, blit=True)

plt.show()

```

## Лабораторная работа №1

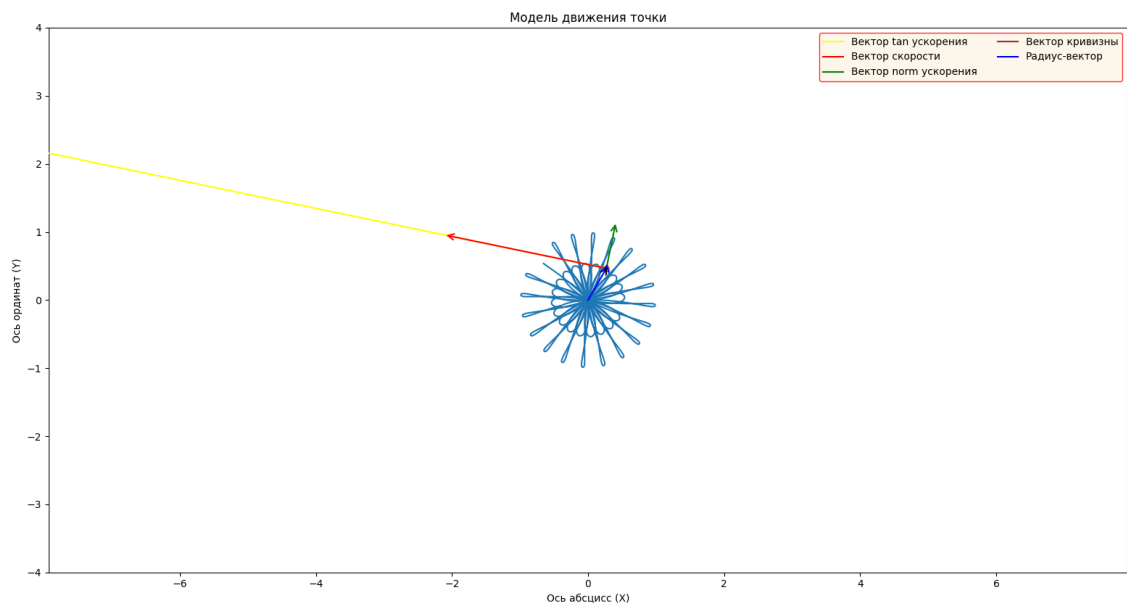
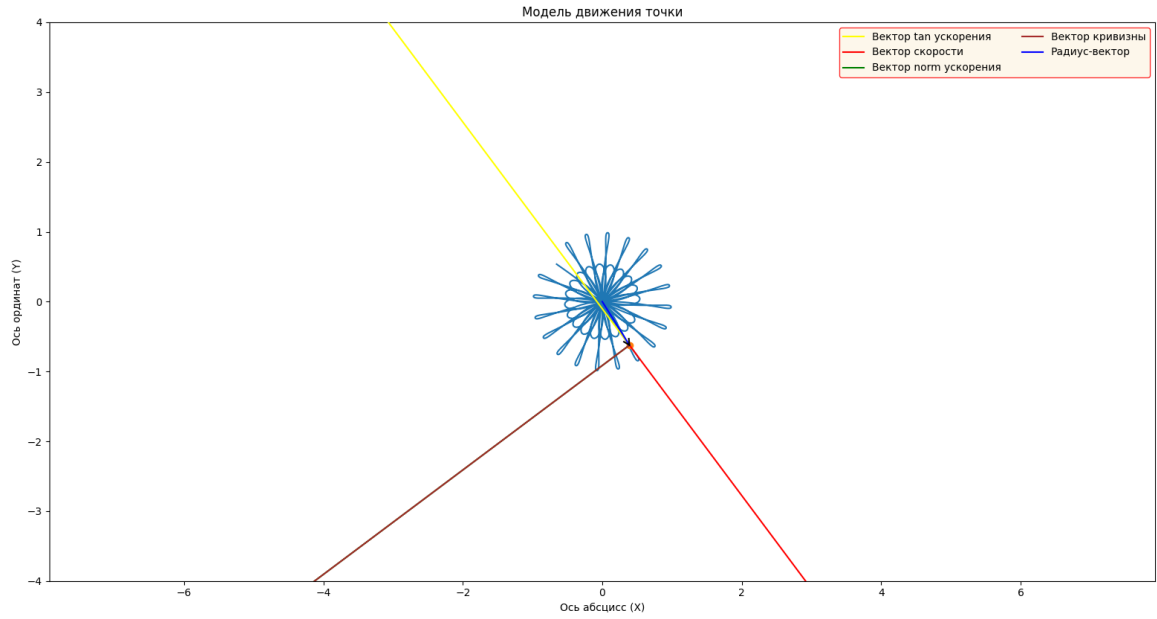
Задание: Смоделировать движение точки которое задано уравнениями:

$$r(t) = 2 + \sin 12t$$

$$\varphi(t) = 1.8t + 0.2 \cos 12t$$

Нарисовать траекторию и компоненты ускорения.

# Результат работы программы.



Вывод: В результате была реализовано и визуализирована анимация движения точки по определённому закону на языке Python с использованием библиотек sympy для символьных вычислений формул и matplotlib для визуального представления графиков